Testkonzept

Modul: T4INF2003 – Software Engineering I
Kurs: TINF2024
Dozentin: Charlotte Chichlow
Semester: Frühjahr 2026

Projekt: Entwicklung eines Online Aufgaben-Planungssystems

Testkonzept.....1
1. Einleitung..... 2
2. Testziele (Test-Pyramide).....2
3. Testebenen und Testarten im Detail.....2
    3.1 Backend (ASP.NET Core)..... 2
    3.2 Frontend (React)..... 3
    3.3 End-to-End Tests (E2E).....3
    3.4 Nicht-funktionale Tests..... 3
    3.5 Abdeckung zentraler Use Cases.....4
4. Testwerkzeuge & Technologie-Stack..... 5
5. Testumgebungen.....5
6. CI/CD-Integration..... 5
7. Testdaten-Management..... 5
8. Qualitäts-Gates & Abnahmekriterien (Definition of Done).....6
9. Risiken & Mitigations.....6
10. Verantwortlichkeiten..... 6
11. Dokumentation von Testergebnissen und Fehlern..... 7

1. Einleitung

Zweck

Dieses Testkonzept definiert die ganzheitliche Teststrategie für die Web-Applikation, um eine hohe Qualität, Stabilität, Sicherheit und Performanz zu gewährleisten. Es deckt sowohl das Backend (ASP.NET Core) als auch das Frontend (Vite + React + TypeScript) sowie die Interaktion zwischen beiden ab.

Geltungsbereich

- Vollständige Web-App (SPA)
- Backend: ASP.NET Core Web API)
- Frontend: Vite + React 18+ + TypeScript + (React Router, TanStack Query)
- Datenbank (SQL Server 2019+)
- Authentifizierung (JWT, OAuth)
- CI/CD-Pipeline (GitHub Actions)

2. Testziele (Test-Pyramide)

Es gilt der Grundsatz, die Testarten projektbezogen anzuwenden:

Ebene	Ziel	Anteil (idealerweise)	Priorität
Unit Tests	Einzelne Einheiten isoliert testen	70 %	Hoch
Integration Tests	Backend-Integration + API-Contract	20 %	Hoch
Component Tests	React-Komponenten + Hooks	5 %	Mittel
E2E / UI Tests	Ende-zu-Ende-Flows (User-Journey)	5 %	Hoch
Nicht-funktionale Tests	Performance, Security, Accessibility	–	Hoch

3. Testebenen und Testarten im Detail

3.1 Backend (ASP.NET Core)

- **Code-Struktur:** Das Package `test` wird im Code-Projekt verwendet, um Testcode strikt vom Quelltext zu trennen. Das Package test wird im Code-Projekt verwendet, um Testcode strikt vom Quelltext zu trennen. Die Testprojekte folgen der Konvention: `backend/tests/<Projektname>.Tests`
- **Unit Tests**
  - Framework: **xUnit.net**
  - Mocking: **NSubstitute**
  - Assertions: **FluentAssertions**
  - Testfokus: Controller, Services, Repositories, Validators, Middleware, Business Logic
- **Integration Tests**
  - WebApplicationFactory<TProgram> + TestServer
  - InMemory-Datenbank oder **Testcontainers** (Docker) für realistische DB-Tests
  - EF Core DbContext-Tests mit UseInMemoryDatabase oder SQLite
  - **Kritischer Integrationstest (Harte Randbedingung):** Das System muss sicherstellen, dass sich Aufgabenblöcke eines Nutzers aus verschiedenen Organisationen zeitlich nicht überschneiden. Dies validiert die zentrale Geschäftslogik des Planungsalgorithmus.
- **API / Contract Tests**
  - Automatisierte HTTP-Tests mit HttpClient + WebApplicationFactory

3.2 Frontend (React)

- **Unit & Component Tests**
  - Framework: **Vitest** (native Vite-Integration, extrem schnell)
  - Rendering: **@testing-library/react** + @testing-library/user-event
  - API-Mocking: **MSW (Mock Service Worker)** – perfekt für TanStack Query
  - TypeScript-Tests werden automatisch durch tsc --noEmit + Vitest abgedeckt
- **Visual / Snapshot Tests** (optional)
  - **Playwright** Component Testing oder **Chromatic** (Storybook)

Test-Struktur: Frontend-Tests (Unit/Component) sind im Ordner `frontend/src/tests/...` abgelegt.

3.3 End-to-End Tests (E2E)

- **Tool-Empfehlung: Microsoft Playwright** (beste Wahl 2026)
  - Vollständige Browser-Automatisierung (Chromium, Firefox, WebKit)
  - Native TypeScript-Unterstützung
  - Exzellente Integration mit Vite (Dev-Server + Build)
  - Alternative: Cypress (falls Team bereits Erfahrung hat)
- **Testfokus:**
  - Auth-Flows (Login, Register, Refresh Token)
  - Kritische User-Journeys
  - Responsiveness (Mobile / Tablet / Desktop)
  - Error-Handling (Netzwerkfehler, 4xx/5xx)

Test-Struktur: E2E-Tests werden in einem dedizierten Root-Ordner `e2e/tests/...` organisiert.

3.4 Nicht-funktionale Tests

- **Performance & Load**
  - Backend: **k6** oder **NBomber**
  - Frontend: Lighthouse CI + Playwright Tracing
- **Security**
  - OWASP ZAP (automatisiert in CI)
  - Backend: **Microsoft.AspNetCore.Authentication** + SecurityHeaders
  - Dependency Scanning: **Trivy** oder **Dependabot** + **dotnet list package --vulnerable**
- **Accessibility (a11y)**
  - **axe-core** + **@playwright/test** oder **jest-axe**
- **SEO / Lighthouse** (bei Public-Seiten)

3.5 Abdeckung zentraler Use Cases

Die folgenden Use Cases aus dem Pflichtenheft werden durch geplante Tests auf verschiedenen Ebenen abgedeckt. Ähnliche Use Cases folgen auch diesem Muster.

UC / Bereich	Testebene	Beispiel Testfälle
4.4 Registrierung	Unit, System/E2E	Gültiger Code → Konto wird angelegt; ungültiger Code; schwaches Passwort; Passwort-Mismatch
4.5 Anmeldung / 4.6 Abmeldung	Unit, System/E2E	Erfolgreicher Login; falsches Passwort; gesperrter User; Logout beendet Session
4.9/4.12 Aufgaben anlegen/verwalten	Unit, Integration	Pflichtfelder, Deadline in Vergangenheit, Statuswechsel, Filter/Sortierung
4.10 Arbeitsplan generieren	Integration, E2E	Keine Aufgaben; mehrere Aufgaben; Konflikt über Organisationen hinweg; Grenzen Arbeitszeit/Belastbarkeit
4.13 Plan anzeigen/anpassen	Integration, E2E	Wochenansicht, verschieben innerhalb zulässiger Zeiten, Verhinderung von Überlappungen
4.7 Arbeitsprofil konfigurieren	Unit, Integration	Ungültige Belastbarkeit, gültige Konfiguration, Auswirkung auf generierten Plan
4.2/4.14 Organisation verwalten	System/E2E	Einladung mit gültiger/ungültiger E-Mail; Entfernen von Mitgliedern; Versuch, letzten Organisierenden zu entfernen (Fokus nur auf MVP-Funktionen)
4.22 Transparente Planung	System/E2E	Anzeige von Parametern bei vorhandenem Plan, Fehlermeldung ohne

		Plan(Optional, ggf. spätere Iteration)

4. Testwerkzeuge & Technologie-Stack

Bereich	Tool / Framework	Bemerkung
Backend Unit/Int	xUnit + NSubstitute + FluentAssertions + WebApplicationFactory	Standard
Frontend	Vitest + Testing Library + MSW	Vite-native
E2E	Playwright (TS)	Empfohlen
API Testing	Playwright	CI-fähig
Coverage	Coverlet (Backend) + V8 Coverage (Vitest)	≥ 80 % Ziel
Reporting	Allure oder built-in Playwright HTML + xUnit XML	CI-Dashboard
Test Data	Testcontainers + Bogus (Fake Data)	Realistisch

5. Testumgebungen

1. **Local** – Entwickler-PC (Docker Compose)
2. **CI/Test** – Automatisierte Pipeline (GitHub Actions)
3. **Production** – Smoke-Tests nach Deployment

6. CI/CD-Integration

- **Pull Request**
  - Backend: dotnet test + Coverage
  - Frontend: vitest run + Type-Check + Build
  - E2E: Playwright gegen Testumgebung (Headless)
- **Dev**
  - Load Tests + Accessibility + Full E2E
- **Main / Release Branch**
  - Vollständige Suite + Security Scan + Performance Baseline

7. Testdaten-Management

- **Bogus** für synthetische Daten
- **Testcontainers** für echte Datenbanken in Integration/E2E
- Anonymisierte Produktionsdaten (nur in Staging)
- Seed-Skripte pro Testklasse (IClassFixture)

8. Qualitäts-Gates & Abnahmekriterien (Definition of Done)

- Angestrebter Unit + Integration Coverage ≥ 80 %
- Keine fehlgeschlagenen Tests in CI auf kritischen Kernpfaden
- Alle kritischen Kernpfade müssen durch E2E-Tests abgedeckt sein.
- Keine kritischen Security- oder a11y-Fehler
- Angestrebter Lighthouse Performance Score ≥ 90 (Desktop)
- Vor Abgabe keine bekannten schwerwiegenden Fehler (High/Critical Bugs) in den Kernfunktionen

9. Risiken & Mitigations

Risiko	Mitigation
Komplexe Auth-Flows	Frühe E2E-Tests + Contract Tests
Frontend-Backend-Drift	MSW + Pact Contracts
Langsame E2E-Tests	Parallelisierung + Selective Testing
TypeScript vs. C# DTOs	OpenAPI (Swagger) + HeyApi

10. Verantwortlichkeiten

- **Backend-Entwickler**: Unit + Integration Tests

  Erstellung von Unit- und Integration Tests im Backend. Verantwortlich für die Einhaltung der Code Coverage (Coverlet).

- **Frontend-Entwickler**: Component + Vitest Tests

  Erstellung von Component- und Vitest Tests für das UI.

- **QA / Test Engineer**: E2E, nicht-funktionale Tests, Testkonzept-Pflege

  Hauptverantwortlich für die Erstellung, Ausführung und Wartung der E2E-Tests (Playwright), nicht-funktionale Tests und die Pflege des Testkonzepts.

- **DevOps**: CI/CD-Pipelines + Test-Umgebungen

  Konfiguration und Wartung der CI/CD-Pipelines (GitHub Actions) und der Test-Umgebungen.

11. Dokumentation von Testergebnissen und Fehlern

Prozess und Zeitpunkt

Die Testdurchführung ist eng an den agilen Sprint-Ablauf gekoppelt:

- Unit- und Integrationstests werden kontinuierlich während der Entwicklung erstellt und lokal ausgeführt.
- Die gesamte automatische Testsuite (Unit, Integration, E2E) wird bei jedem **Pull Request** auf einen Hauptbranch (z. B. `main`) als Quality Gate ausgeführt.
- Manuelle Systemtests werden am Ende des Sprints vor dem Deployment in die `Dev`- oder `Production`-Umgebung durchgeführt.

Ergebnisdokumentation

- Automatisierte Tests: Durchgeführte Tests werden in der CI/CD-Pipeline (GitHub Actions) und im zugehörigen Pull Request dokumentiert. Wir nutzen das [Allure oder built-in Playwright HTML + xUnit XML] Reporting, um eine Dashboard-Ansicht der Ergebnisse zu erhalten.
- Fehlererfassung: Gefundene Fehler werden als Issues/Tickets in unserem Projektboard (GitHub Issues) erfasst und enthalten stets: Reproduktionsschritte, Erwartetes Verhalten, Tatsächliches Verhalten und Priorität (High/Critical/Low).
- Review-relevante Testfälle: Für die Abnahme kritischer Kernfunktionen (z.B. Arbeitsplan-Generierung) wird zusätzlich eine kurze, manuelle Testfallliste im Projekt-Wiki (oder einem dedizierten Board-Abschnitt) gepflegt, die vor dem finalen Review durch den QA/Test Engineer abgehakt wird.